

Отчёт по тестовому заданию

## Обнаружение 68 ключевых точек лица

Автор: Юрий Языков

1 июня 2026 г.

# Содержание

|           |                                                   |           |
|-----------|---------------------------------------------------|-----------|
| <b>1</b>  | <b>Введение</b>                                   | <b>2</b>  |
| <b>2</b>  | <b>Постановка задачи</b>                          | <b>2</b>  |
| <b>3</b>  | <b>Предобработка</b>                              | <b>2</b>  |
| <b>4</b>  | <b>Архитектура моделей</b>                        | <b>2</b>  |
| <b>5</b>  | <b>Процесс обучения</b>                           | <b>2</b>  |
| 5.1       | Основной ход обучения . . . . .                   | 2         |
| 5.2       | Обработка данных . . . . .                        | 2         |
| 5.3       | Функции потерь . . . . .                          | 3         |
| 5.4       | Функция потерь с графовым лапласианом . . . . .   | 3         |
| 5.5       | Гиперпараметры . . . . .                          | 4         |
| <b>6</b>  | <b>Визуальный результат работы моделей</b>        | <b>5</b>  |
| <b>7</b>  | <b>Кривые обучения в логарифмическом масштабе</b> | <b>6</b>  |
| <b>8</b>  | <b>CED на датасете 300W</b>                       | <b>7</b>  |
| <b>9</b>  | <b>CED на датасете Менро</b>                      | <b>8</b>  |
| <b>10</b> | <b>Сравнение моделей</b>                          | <b>8</b>  |
| <b>11</b> | <b>Анализ результатов</b>                         | <b>9</b>  |
| <b>12</b> | <b>Заключение</b>                                 | <b>9</b>  |
| <b>13</b> | <b>Возможные улучшения</b>                        | <b>9</b>  |
| <b>A</b>  | <b>Инструкция по запуску</b>                      | <b>10</b> |

# 1 Введение

Задание заключается в реализации алгоритма обнаружения 68 особых точек на лице человека (face alignment), тестировании данного алгоритма на общедоступных датасетах и сравнении с аналогами.

## 2 Постановка задачи

Исследовать различные архитектуры желательно до 60 мб и получить результат лучший чем модель из библиотеки dlib

## 3 Предобработка

Не получилось подружить dlib с cuDNN так что была использована обычная реализация поиска лиц через HOG из библиотеки dlib реализация в файле *Preproces\_mult.py* Для аугментаций использовались только преобразования по цвету, т.к. преобразования пространства с ходу бы не запустились из библиотек Albumentations или torchvision, можно было реализовать самостоятельно через реализацию поворота сдвига и т.п. чтобы также сдвигать и размеченные точки, но т.к. backbone предобучен по imagenet с аугментациями это бы не дало весомого прироста, а время затраченное на решение ошибок с матрицами могло быть высоким.

## 4 Архитектура моделей

Рассмотрены backbone от resnet 18, Efficientnet B2,B3, Convnext, mobilenet,mobilevit В основе своей во всех моделях брался backbone + добавлялся двуслойный линейный слой для предсказания точек. подробнее можно посмотреть в *models.py*

## 5 Процесс обучения

### 5.1 Основной ход обучения

Первоначально хотелось получить какой-то бейзлайн была взята MSE как в статье для координат точек, и проверены 3 модели с resnet18,mobilenetv3,Efficient<sub>n</sub>etB2DLIB, LR5.SMOOTH. 0.07 результат получился более чем хорошим.

### 5.2 Обработка данных

Изначальное предположение о то что датасет используется в коммерции, значит ошибок разметки не так много и отсутствовало желание просматривать точки самостоятельно, в целом в ходе обучения никаких проблем свидетельствовавших о большом количестве шума в данных или плохой разметке не было, также за счет scheduler'а у модели было меньше шансов начать обучаться под шум. В дальнейшем скорее всего стоит реализовать аугментации с поворотом, и попробовать поиграться с набором

данных возможно также будет прирост.

В ходе препроцессинга из трейна выкидывались bbox у которых IOU с разметкой меньше чем 0.3, а также добавлялись по 15 процентов по бокам bbox, т.к. для сверточной сети от этого хуже не станет.

Использовались только цветовые аугментации и приведение к нормализованному виду относительно imagenet

### 5.3 Функции потерь

Исследованы функции MSE, SmoothL1loss, кастомная лосс функция смесь MSE с учетом связности лапласиана графа. Кастомная лосс функция имеет лучший результат, в целом учитывает форму предсказанных точек. Ниже дано ее математическое представление

### 5.4 Функция потерь с графовым лапласианом

Для учета локальной геометрической структуры лица каждая из 68 ключевых точек рассматривается как вершина графа

$$G = (V, E),$$

где множество вершин  $V$  соответствует ландмаркам лица, а множество ребер  $E$  задает связи между соседними точками (контур лица, брови, глаза, нос и губы).

Матрица смежности графа определяется как

$$A_{ij} = \begin{cases} 1, & \text{если вершины } i \text{ и } j \text{ соединены ребром,} \\ 0, & \text{иначе.} \end{cases}$$

Для каждой вершины вычисляется ее степень

$$d_i = \sum_j A_{ij},$$

после чего формируется диагональная матрица степеней

$$D = \text{diag}(d_1, d_2, \dots, d_{68}).$$

Графовый лапласиан определяется как

$$L = D - A.$$

Пусть

$$X = \begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \\ \vdots & \vdots \\ x_{68} & y_{68} \end{bmatrix}$$

обозначает координаты всех ландмарок лица.

Тогда действие лапласиана на вершину  $i$  имеет вид



$$(LX)_i = d_i X_i - \sum_{j \in N(i)} X_j,$$

где  $N(i)$  — множество соседей вершины  $i$ .

Полученная величина характеризует локальное отклонение точки от положения ее соседей и отражает геометрическую форму соответствующего контура лица.

Для предсказанных координат  $X_p$  и эталонных координат  $X_t$  вычисляются лапласианные представления

$$\tilde{X}_p = LX_p, \quad \tilde{X}_t = LX_t.$$

Лапласианный штраф определяется как среднеквадратичная ошибка между ними:

$$L_{\text{lap}} = \frac{1}{68} \sum_{i=1}^{68} \left\| (\tilde{X}_p)_i - (\tilde{X}_t)_i \right\|^2.$$

Базовой функцией потерь является среднеквадратичная ошибка координат:

$$L_{\text{base}} = \frac{1}{68} \sum_{i=1}^{68} \left\| (X_p)_i - (X_t)_i \right\|^2.$$

Итоговая функция потерь определяется выражением

$$L = L_{\text{base}} + \alpha L_{\text{lap}},$$

где  $\alpha$  является гиперпараметром, регулирующим вклад лапласианной регуляризации.

Таким образом, минимизация данной функции потерь обеспечивает не только точное восстановление координат отдельных ключевых точек, но и сохранение локальной геометрической структуры лица.

## 5.5 Гиперпараметры

| Параметр      | Значение |
|---------------|----------|
| Batch Size    | 64       |
| Optimizer     | AdamW    |
| Learning Rate | 1e-4     |
| Epochs        | 100 160  |
| Image Size    | 224      |

Таблица 1: Параметры обучения

## 6 Визуальный результат работы моделей



(a) DLIB



(b) Convnext tiny GraphLaplacian

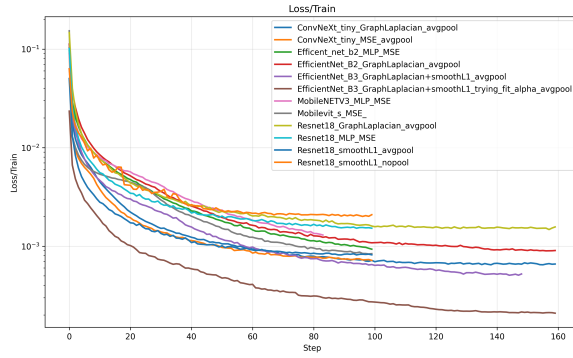


(c) Resnet18 GraphLaplacian

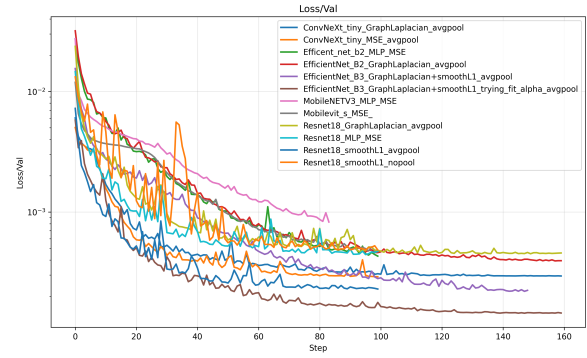


(d) Effnet 3 alpha 0.07

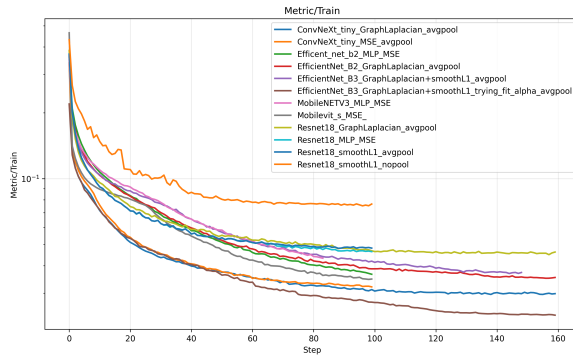
## 7 Кривые обучения в логарифмическом масштабе



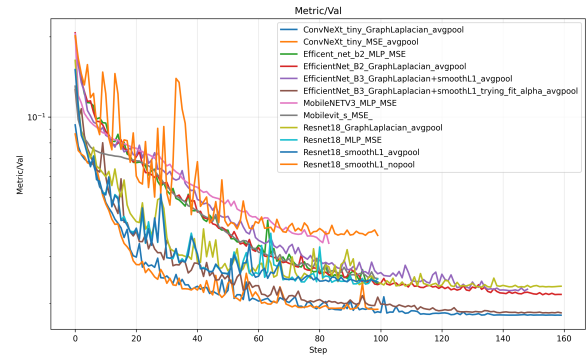
(a) Train Loss



(b) Validation Loss



(c) Train metric



(d) Validation metric

Рис. 2: Кривые обучения модели

## 8 CED на датасете 300W

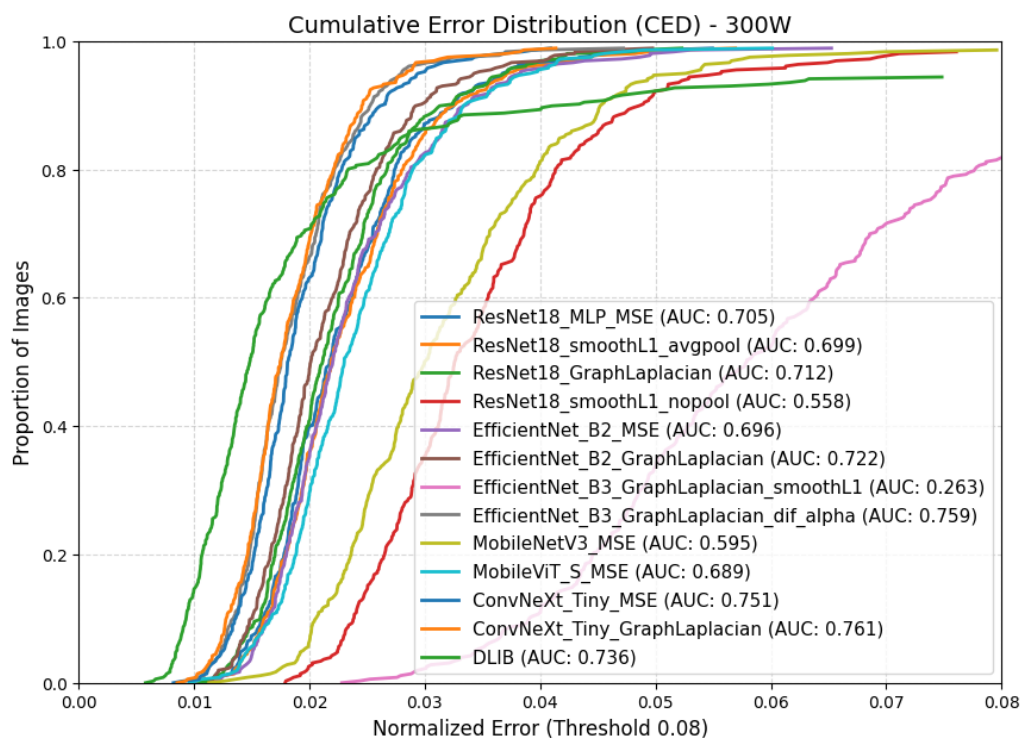


Рис. 3: CED-кривая для датасета 300W

## 9 CED на датасете Менпо

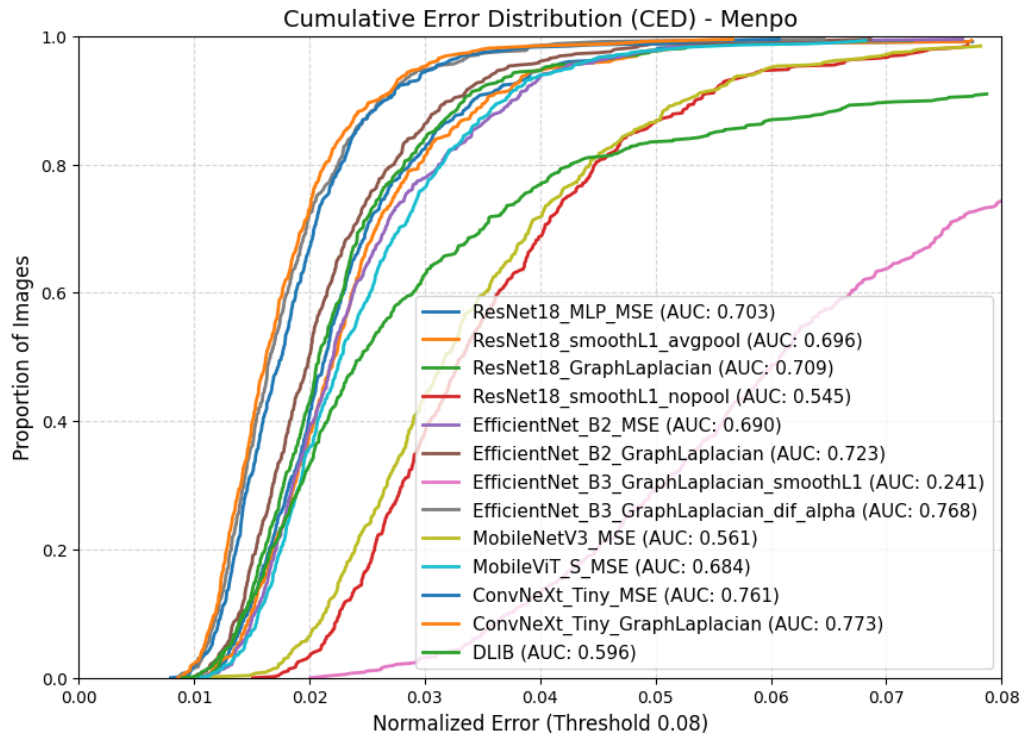


Рис. 4: CED-кривая для датасета Менпо

## 10 Сравнение моделей

| Модель                                                  | Вес, Мб | 300W AUC | Menpo AUC |
|---------------------------------------------------------|---------|----------|-----------|
| ResNet18 MSE                                            | 44      | 0.7045   | 0.7033    |
| ResNet18 GraphLaplacian                                 | 44      | 0.7121   | 0.7088    |
| ResNet18 SmoothL1                                       | 44      | 0.6994   | 0.6959    |
| ResNet18 SmoothL1 (NoPool)                              | 56      | 0.5579   | 0.5450    |
| EfficientNet B2 MSE                                     | 32      | 0.6957   | 0.6904    |
| EfficientNet B2 GraphLaplacian                          | 32      | 0.7225   | 0.7228    |
| EfficientNet B3 GraphLaplacian SmoothL1loss(alpha=0.2)  | 44      | 0.2627   | 0.2409    |
| EfficientNet B3 GraphLaplacian SmoothL1loss(alpha=0.07) | 44      | 0.7585   | 0.7677    |
| MobileNetV3 MSE                                         | 13      | 0.5948   | 0.5611    |
| MobileViT-S MSE                                         | 20      | 0.6886   | 0.6842    |
| ConvNeXt-Tiny MSE                                       | 109     | 0.7513   | 0.7610    |
| ConvNeXt-Tiny GraphLaplacian                            | 109     | 0.7613   | 0.7733    |
| DLIB                                                    | 98      | 0.7363   | 0.5961    |

Таблица 2: Сравнение моделей

## 11 Анализ результатов

Полученные результаты показывают, что использование современных архитектур позволяет достичь качества, сопоставимого либо превосходящего DLIB.

Наилучший результат продемонстрировала модель CONVNEXT Tiny with graphlaplasian loss но она больше 60 мб среди представителей менее 60 мб первое место у EFFICIENT NET V3

## 12 Заключение

В ходе работы были исследованы несколько архитектур моделей, а также несколько loss функций для задачи обнаружения 68 ключевых точек лица. Было показано что современные архитектуры лучше справляются с этой задачей, а также была найдена не стоковая Loss функция которая улучшила поиск оптимума smoothl1+graphlaplasian.

## 13 Возможные улучшения

Важно отметить, что текущий ресерч, вероятно, является лишь первой итерацией поиска решения, близкого к современному уровню качества.

Среди возможных направлений дальнейшего улучшения можно выделить следующие:

- Среди архитектур размером менее 60 Мб был найден явный фаворит, однако исследование нельзя считать исчерпывающим. Не все современные архитектуры классификаторов были протестированы. Перспективным направлением является исследование моделей с механизмами attention, а также использование более сложных многослойных голов предсказания.
- В графовой части функции потерь взаимодействие рассматривается лишь между двумя соседними вершинами. В дальнейшем можно исследовать влияние более дальних связей, например учитывать две или три соседние точки с каждой стороны, а также использовать различные схемы взвешивания рёбер графа.
- Также представляет интерес исследование более сложных графовых структур, учитывающих взаимосвязи между различными частями лица, а не только внутри отдельных групп особых точек.
- Дополнительным направлением является создание аугментаций + поиск изображений с истинно плохой разметкой, которые мешают обучению, возможным решением будет либо добавить вес изображениям либо просто выкинуть их из датасета, а также проверка мешают ли они обобщающим способностям модели или нет при помощи обучения на датасете с этим и без этих данных и проверке например 0 гипотезы о значимости на тестовой части, если они будут значимо мешать или улучшать обучение, можно будет принимать решение,но это также требует доп ресерча.

## А Инструкция по запуску

сам код на репозитории [https://github.com/pechenie8v8kletku/Yazikov\\_Iurii](https://github.com/pechenie8v8kletku/Yazikov_Iurii)  
+ *tensorboard* [https : //disk.360.yandex.ru/d/XK4EY2rFxyS9A](https://disk.yandex.ru/d/XK4EY2rFxyS9A)

Препроцессинг:

```
python Preproces_mult.py
```

Обучение:

```
python main.py
```

Тестирование:

```
python test.py
```